TP2
STM32F446RE I2C Communication
with ADC

Filière:

2ème Années Mécatronique & Technologies Automobiles

Supervised by:
Pr. Wissam JENKAL
Mr. Rachid AMENZOUY

STM32CubeMX • HAL Library • Experimental LAB

0 Contents

1 Project Overview

This guide implements the **I2C** communication between two STM32F446RET6 boards:

- **Master board** performs an I2C read request for 2 bytes and reconstructs the ADC value. The result is checked via debugger.
- **Slave board** runs in *I2C slave* mode. It continuously samples a potentiometer via its 12-bit ADC. When the master sends the read request, the slave prepares the ADC reading and transmit it as 2-byte ADC value.

1.1 I2C Key Concepts

Term	Meaning
SDA	Serial Data line — bidirectional, open-drain
SCL	Serial Clock line — driven by master, open-drain
7-bit address	Unique address per slave on the same bus (here `0x27`)
START / STOP	Conditions generated by master to begin/end a transaction
ACK / NACK	Acknowledge bit sent by receiver after each byte
Write transaction	Master sends address + W bit, then data bytes
Read transaction	Master sends address + R bit, slave sends data bytes
Pull-up resistors	Required on SDA and SCL (4.7 kΩ to 3.3 V)

2 Hardware Requirements

Component	Details
STM32F446RET6	× 2 — Master and Slave
Potentiometer (10 kΩ)	Wiper connected to slave ADC input (PA0)
Pull-up resistors	4.7 kΩ × 2 (one for SDA, one for SCL) to 3.3 V
Jumper wires	I2C bus + power
3.3 V power supply	Common GND between boards

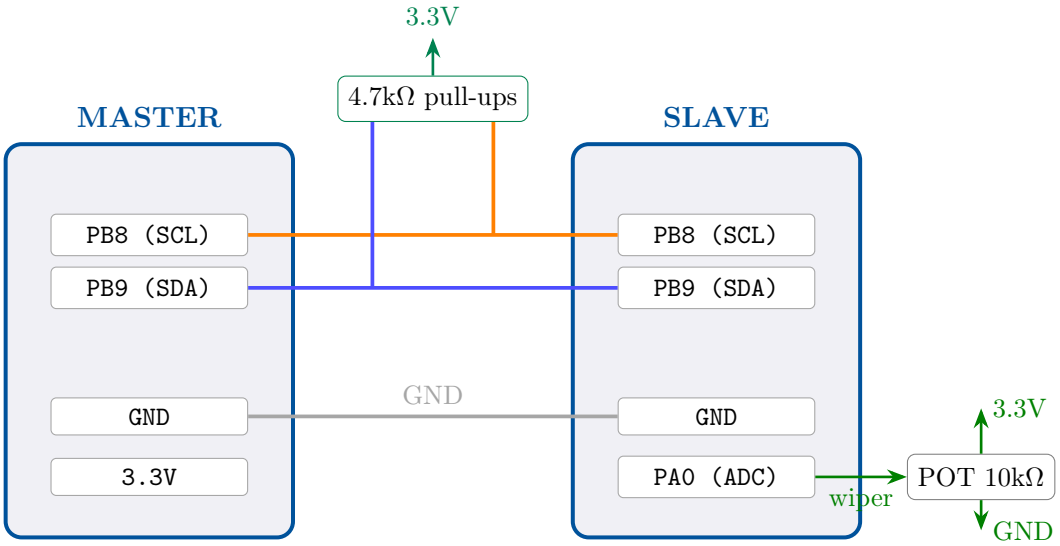
Pull-up resistors are mandatory! I2C lines are open-drain. Without 4.7 kΩ pull-ups from SDA and SCL to 3.3 V the bus will not function. One pair of pull-ups is sufficient for the entire bus regardless of how many devices are connected.

3 Pin Mapping and Wiring

3.1 I2C1 Pin Assignments (Both Boards)

STM32F446RE I2C1 default alternate function pins:

Signal	Master Pin	Slave Pin	Notes
SCL	PB8	PB8	Open-drain, shared bus
SDA	PB9	PB9	Open-drain, shared bus



3.2 ADC Pin (Slave Only)

Signal	Slave Pin	Notes
ADC input	PA0 (ADC1_IN0)	Center wiper of potentiometer
VCC ref	3.3 V	Potentiometer top terminal
GND ref	GND	Potentiometer bottom terminal

3.3 Experimental setup

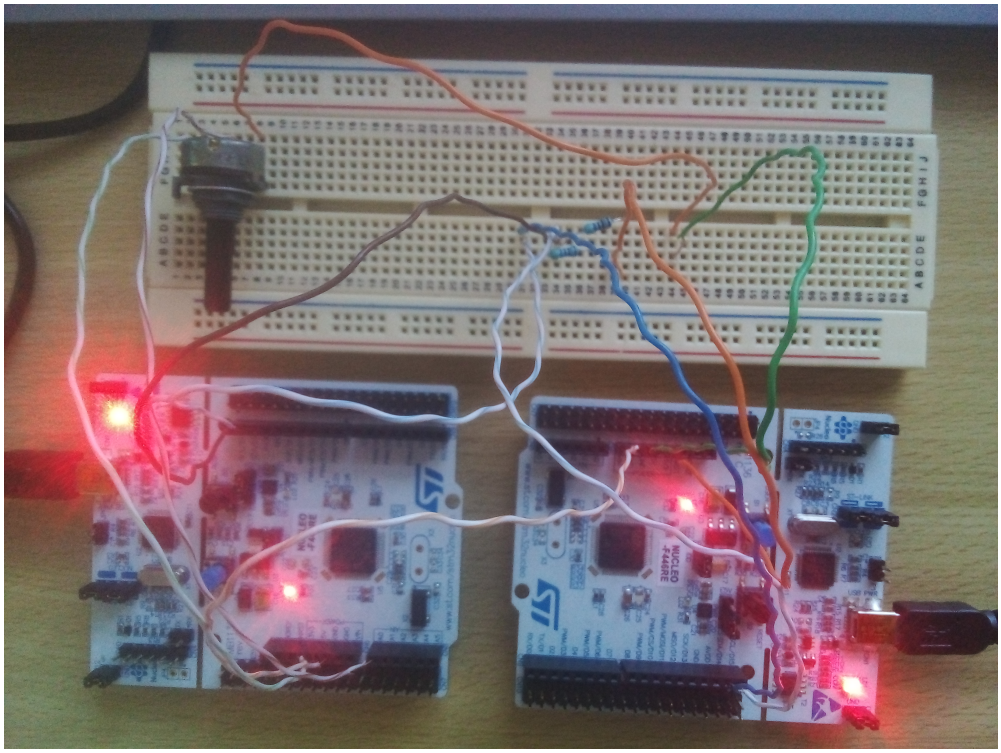


Figure 1: Experimental setup.

4 STM32CubeMX Configuration

4.1 Master Board Configuration

Create a new project at STM32CubeMX or STM32CubeIDE for the Master board.

4.1.1 I2C1 — Master

Navigate to **Connectivity** → **I2C1** and apply:

Parameter	Value
I2C Mode	I2C
Speed Mode	Standard Mode
Speed Frequency (kHz)	100
Fast Mode Duty Cycle	(not applicable in Standard Mode)
Slave Address Length	(master does not need one)
Own Address 1	0x00 (unused for master)
Dual Address Acknowledged	Disabled
General Call Address Detection	Disabled
No Stretch Mode	Disabled

CubeMX will automatically assign **PB8** as SCL and **PB9** as SDA in open-drain alternate function mode.

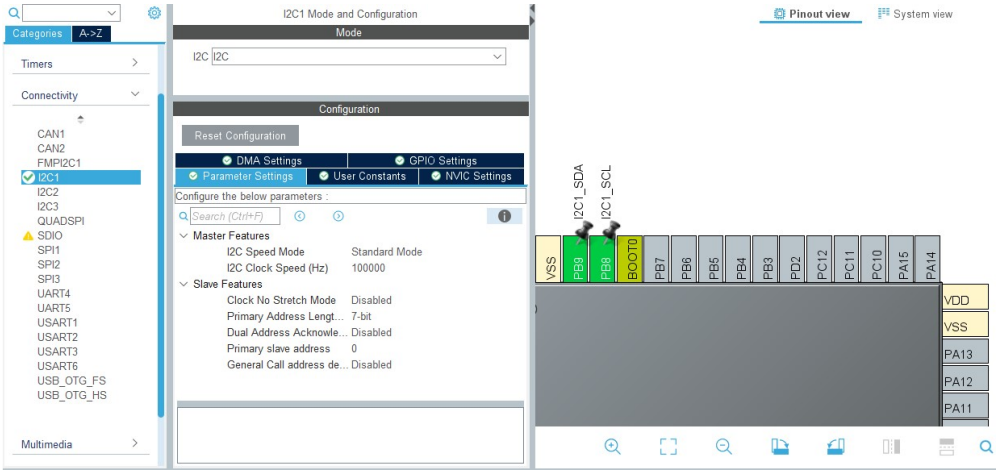


Figure 2: I2C Master Configuration.

Save the configuration: After finishing the configuration click **Ctrl** + **S**.

4.2 Slave Board Configuration

Create a new project at STM32CubeMX or STM32CubeIDE for the Slave board.

4.2.1 I2C1 — Slave

Navigate to **Connectivity** → **I2C1**:

Parameter	Value
I2C Mode	I2C
Speed Mode	Standard Mode
Speed Frequency (kHz)	100
Own Address 1	**0x13** (7-bit, left-shifted = `0x26` in HAL)
Dual Address Acknowledged	Disabled
General Call Address Detection	Disabled
No Stretch Mode	**Disabled** (clock stretching must be ON for slave)

HAL address convention: The STM32 HAL I2C functions expect the 7-bit address *left-shifted by 1*. So address `0x13` is passed as `(0x13 « 1)` = `0x26` in HAL master calls. On the slave side, CubeMX stores the address directly as `0x13` in the Own Address field — the peripheral handles the shift internally.

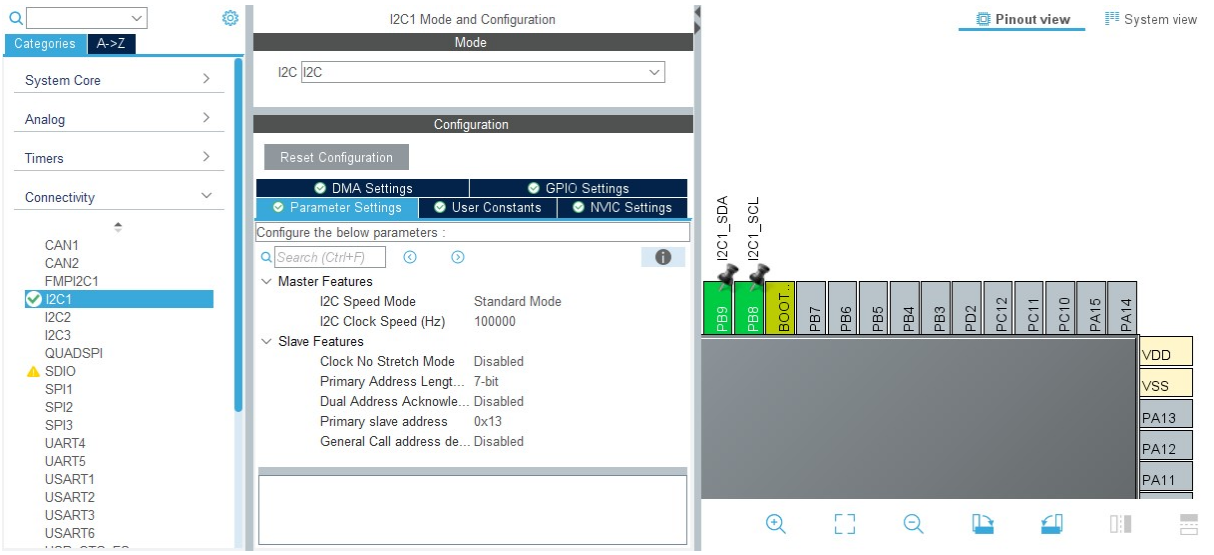


Figure 3: I2C Slave Configuration.

4.2.2 NVIC — Slave (I2C Interrupts)

Navigate to **System Core** → **Connectivity** → **I2C1** → **NVIC** and enable:

- **I2C1 event interrupt**
- **I2C1 error interrupt**

The slave firmware uses **interrupt mode** (`_IT` HAL functions) because a slave cannot predict when the master will address it. Polling mode would block the main loop indefinitely. Interrupts allow the ADC to keep running while the slave waits.

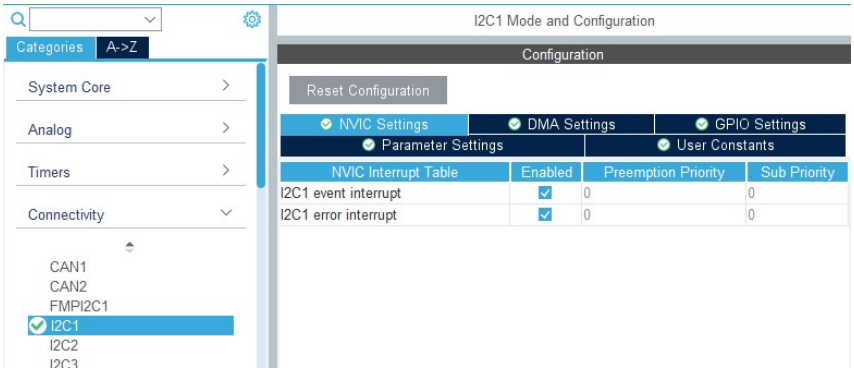


Figure 4: I2C Slave Interrupts.

4.2.3 ADC1 — Slave

Navigate to **Analog** → **ADC1**:

Parameter	Value
IN0	Single-ended (channel 0 → PA0)
Resolution	12 Bits
Data Alignment	Right
Scan Conversion Mode	Disabled
Continuous Conversion Mode	Enabled
External Trigger	Software Start
Sampling Time	239.5 cycles

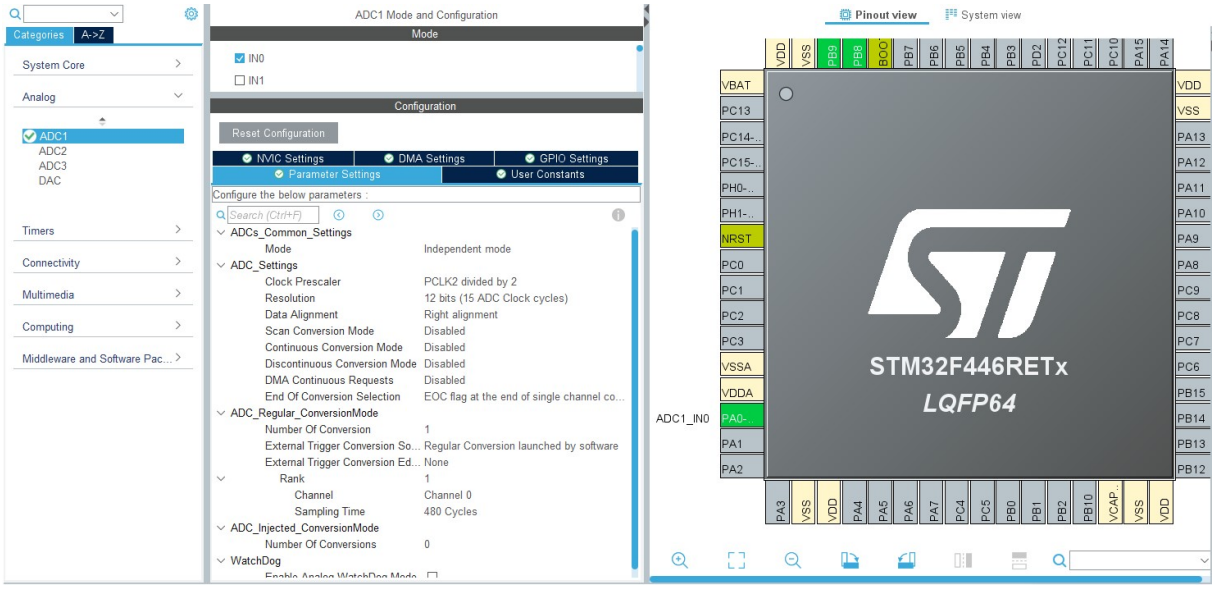


Figure 5: Slave ADC Configuration.

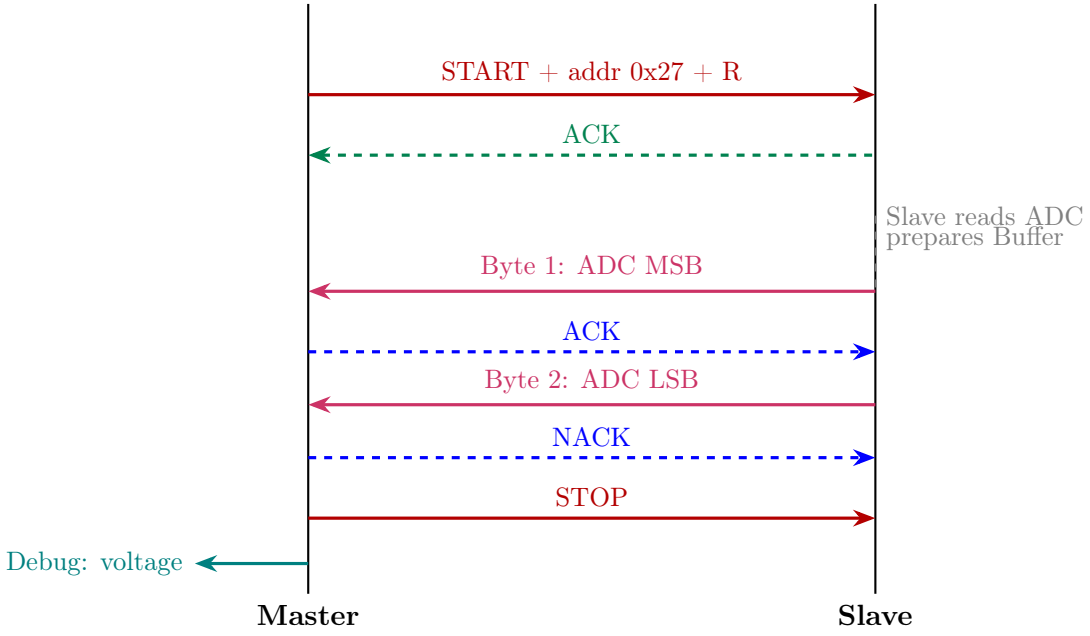
Save the configuration: After finishing the configuration click **Ctrl + S**.

5 Communication Protocol

5.1 Transaction Sequence

I2C transactions always follow the pattern: **START → Address+R/W → Data → STOP**.

In this application Master requests 2 bytes from slave address `0x13`. Slave transmits the buffered ADC MSB then LSB.



5.2 ADC Value Encoding

The 12-bit ADC value (0–4095) is split across two bytes:

$$\text{tx\_buf}[0] = \frac{\text{adc\_val}}{256}\ \text{(MSB)}, \qquad \text{tx\_buf}[1] = \text{adc\_val} \bmod 256\ \text{(LSB)}$$

Reconstruction on the master:

$$\text{adc\_raw} = (\text{rx\_buf}[0] \ll 8) \mid \text{rx\_buf}[1]$$

Voltage:

$$V = \frac{\text{adc\_raw}}{4095} \times 3.3\,\text{V}$$

6 Master Firmware

6.1 Complete `main.c` — Master

```c
/* USER CODE END Header */
/* Includes
    ------------------------------------------------------------------
    */
#include "main.h"

/* Private includes
    ----------------------------------------------------------*/
/* USER CODE BEGIN Includes */
#include <stdio.h>
#include <string.h>
/* USER CODE END Includes */

/**
* @brief  The application entry point.
* @retval int
*/
int main(void)
{
  /* USER CODE BEGIN 2 */
  uint8_t  rx_buf[2]  = {0, 0};   /* received ADC bytes  */
  uint16_t adc_raw    = 0;
  float voltage    = 0.0f;
  uint16_t SLAVE_ADDR  = (0x13 << 1);   /* = 0x26 */

  /* USER CODE END 2 */

  /* Infinite loop */
  /* USER CODE BEGIN WHILE */
  while (1)
  {
    HAL_I2C_Master_Receive(&hi2c1, SLAVE_ADDR, rx_buf, 2, 1000);

    adc_raw = ((uint16_t)rx_buf[0] << 8) | rx_buf[1];
    voltage = ((float)adc_raw / 4095.0f) * 3.3f;

    HAL_Delay(500);
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
  }
  /* USER CODE END 3 */
}
```

Listing 1: Master firmware

7 Slave Firmware

7.1 Complete `main.c` — Slave

```c
/* USER CODE END Header */
/* Includes
    ------------------------------------------------------------------
    */
#include "main.h"

/* Private includes
    ----------------------------------------------------------*/
/* USER CODE BEGIN Includes */
#include <stdio.h>
#include <string.h>
/* USER CODE END Includes */

/* USER CODE BEGIN PFP */
static uint16_t ADC_Read(void)
{
  HAL_ADC_Start(&hadc1);
  HAL_ADC_PollForConversion(&hadc1, HAL_MAX_DELAY);
  uint16_t val = (uint16_t)HAL_ADC_GetValue(&hadc1);
  HAL_ADC_Stop(&hadc1);
  return val;
}
/* USER CODE END PFP */

/* Private user code
    ---------------------------------------------------------*/
/* USER CODE BEGIN 0 */
volatile uint8_t cmd_received = 0;
volatile uint8_t tx_done      = 0;
static uint8_t rx_buf[1] = {0};
static uint8_t tx_buf[2] = {0, 0};

void HAL_I2C_AddrCallback(I2C_HandleTypeDef *hi2c,
uint8_t TransferDirection,
uint16_t AddrMatchCode)
{
  if (TransferDirection == I2C_DIRECTION_TRANSMIT)
  {
    /* Master writing -> Slave receives */
    HAL_I2C_Slave_Seq_Receive_IT(hi2c, rx_buf, 1,
       I2C_FIRST_AND_LAST_FRAME);
  }
  else
  {
    /* Master reading -> Slave sends */
    HAL_I2C_Slave_Seq_Transmit_IT(hi2c, tx_buf, 2,
       I2C_FIRST_AND_LAST_FRAME);
  }
}

```

```
/* Called when slave has fully received 'Size' bytes from master */
void HAL_I2C_SlaveRxCpltCallback(I2C_HandleTypeDef *hi2c)
{
  HAL_I2C_EnableListen_IT(hi2c);
}

/* Called when slave has fully transmitted 'Size' bytes to master */
void HAL_I2C_SlaveTxCpltCallback(I2C_HandleTypeDef *hi2c)
{
  HAL_I2C_EnableListen_IT(hi2c);
}

void HAL_I2C_ListenCpltCallback(I2C_HandleTypeDef *hi2c)
{
  HAL_I2C_EnableListen_IT(hi2c);
}

void HAL_I2C_ErrorCallback(I2C_HandleTypeDef *hi2c)
{
  if (HAL_I2C_GetError(hi2c) == HAL_I2C_ERROR_AF ||
  HAL_I2C_GetError(hi2c) == HAL_I2C_ERROR_BERR)
  {
    HAL_I2C_EnableListen_IT(hi2c);
  }
}
/* USER CODE END 0 */

/**
* @brief  The application entry point.
* @retval int
*/
int main(void)
{
  /* USER CODE BEGIN 2 */
  uint16_t adc_val;
  HAL_I2C_EnableListen_IT(&hi2c1);
  /* USER CODE END 2 */

  /* Infinite loop */
  /* USER CODE BEGIN WHILE */
  while (1)
  {
    adc_val = ADC_Read();
    tx_buf[0] = (uint8_t)(adc_val >> 8);
    tx_buf[1] = (uint8_t)(adc_val & 0xFF);

    HAL_Delay(500);

    /* USER CODE END WHILE */
```

```
94
95      /* USER CODE BEGIN 3 */
96    }
97    /* USER CODE END 3 */
98  }
```

Listing 2: Slave firmware

8 Project Debugging and Expected Output

After completing the project configuration, connect the STM32F446RE board then upload your Slave application to the board. Next, disconnect the first board and connect the second STM32F446RE board. Then upload your Master application. Finally, connect both boards then debug the Master application.

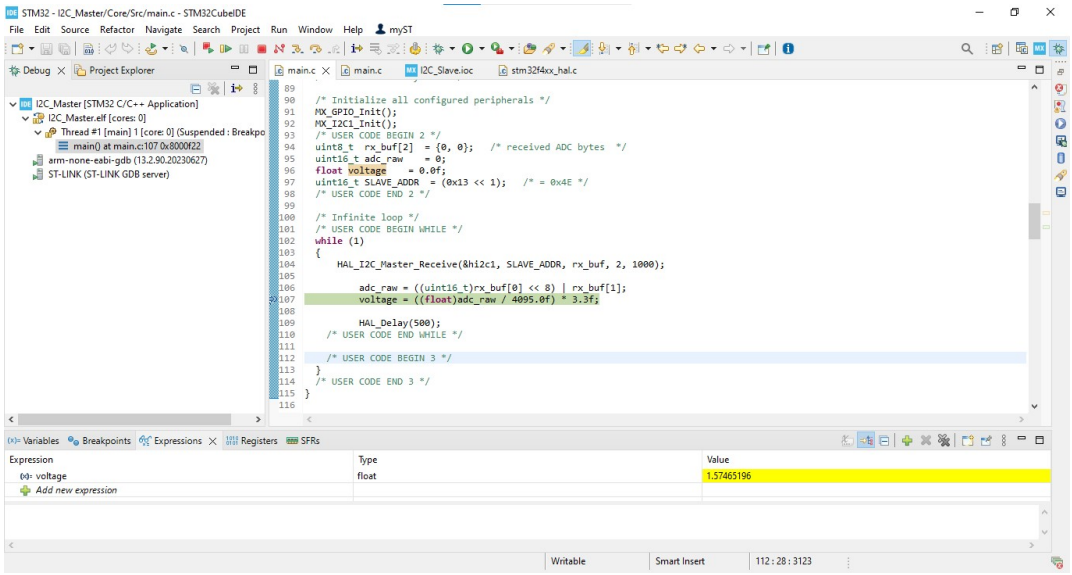


Figure 6: Project Debugging.

Put a break point at **voltage** variable then start debugging, for each run you should notice the changes in **voltage** value.

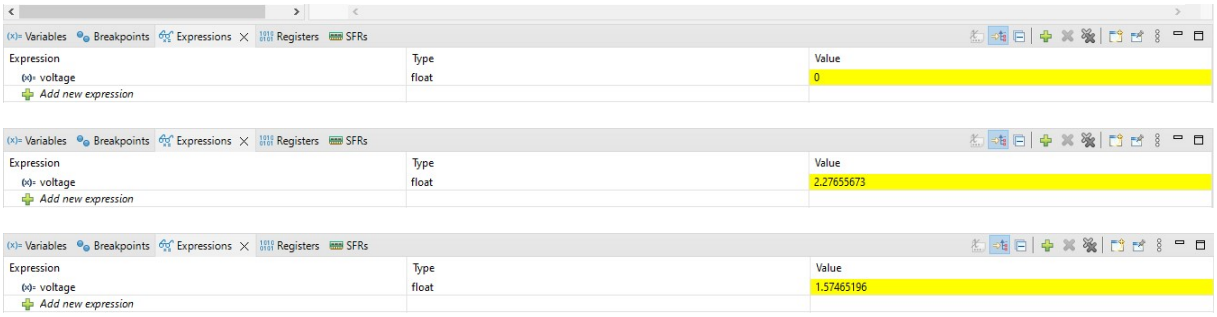


Figure 7: Expected Output.

End of Guide